BINARY-SEARCH-TREE-AND-TREE-SORT

이진 탐색 트리와 트리정렬

연관 기출 · 129-1-13, 137-1-12

한눈에 이해

새 수가 현재 수보다 작으면 왼쪽, 크면 오른쪽으로 내려갑니다. 트리를 왼쪽 서브트리→현재 노드→오른쪽 서브트리 순으로 읽으면 작은 값부터 나옵니다.

정의와 핵심 원리

BST는 각 노드의 왼쪽 서브트리 모든 키가 해당 키보다 작고 오른쪽의 모든 키가 큰 이진 트리입니다. 트리정렬은 입력을 BST에 삽입한 뒤 중위순회로 정렬 결과를 출력하는 방법입니다.

• 불변식은 바로 옆 자식뿐 아니라 서브트리 전체에 적용합니다. 검색은 같으면 종료, 작으면 왼쪽, 크면 오른쪽으로 이동하고, 삽입은 그 탐색이 끝나는 빈 링크에 새 노드를 연결합니다.

• 삭제는 잎이면 링크 제거, 자식이 하나면 자식으로 연결, 두 개면 오른쪽 서브트리 최솟값인 중위 후속자 또는 왼쪽 최댓값인 전임자로 대체하고 원래 위치를 제거합니다. 대체 후에도 순서 조건을 보존해야 합니다.

• 검색·삽입·삭제는 높이 h에 대해 O(h)입니다. 무작위 삽입의 평균은 O(log n)이지만, 정렬된 값을 일반 BST에 넣으면 높이가 n-1로 치우칩니다. AVL·레드블랙 등 균형을 유지해야 로그 성능을 보장할 수 있습니다.

• 트리정렬은 구축 비용과 Θ(n) 순회 비용의 합입니다. 중복은 빈도나 레코드 목록으로 보존해야 합니다. 같은 키를 덮어쓰는 심벌 테이블 코드를 그대로 쓰면 원소가 사라지며, 안정성도 자동 보장되지 않습니다.

구조·비교·판단

구분	시간·공간	핵심 차이
일반 BST 연산	O(h); 최악 O(n)	삽입 순서가 높이를 좌우
일반 BST 트리정렬	평균 O(n log n), 최악 $\Theta(n^2)$	평균은 입력 순서의 가정 필요
균형 BST 트리정렬	최악 O(n log n), 노드 O(n)	균형 유지 비용; 배열 제자리 정렬 아님
일반 이진 트리·힙	BST와 다른 제약	자식 수 제한·부모 자식 우선순위만으로 전체 정렬 불가

적용 예제

7개 키의 구축·검색·정렬·삭제

강사 생성 학습 예제입니다. 공식 기준답안이 아닙니다.

조건: 빈 일반 BST에 [5,3,7,2,4,6,8]을 순서대로 삽입합니다.

1. 5를 루트, 3·7을 좌·우 자식, 2·4를 3 아래, 6·8을 7 아래 둡니다. 새 키마다 방문한 기존 노드 수를 세면 0+1+1+2+2+2+2=10회 키 비교입니다.

2. 6 검색은 5→7→6으로 3개 노드를 방문합니다. 중위순회는 2,3,4,5,6,7,8로 각 키를 한 번 출력합니다.

3. 5 삭제 시 오른쪽 서브트리 최솟값 6으로 대체하고 원래 6을 제거합니다. [1,2,3,4]를 삽입하면 사슬이 되어 구축 비교는 0+1+2+3=6회입니다.

결과·해석: 모든 노드가 좌우 키의 순서 조건을 만족하므로 중위 순회 결과가 정렬됩니다. 탐색에 드는 시간은 트리 높이에 따라 달라집니다.

전제·주의: 비교 횟수는 노드 방문마다 키를 한 번 비교한다는 교육용 모형입니다. 포인터·재귀 연산까지 센 실제 명령 수는 아닙니다.

답안 골격과 혼동 방지

BST 정의·전체 서브트리 조건 → 삽입·검색·삭제 도식 → 중위순회→트리정렬 → h 기반 복잡도와 편향 사례 → 중복 보존·균형·메모리

- BST를 항상 균형 잡힌 트리 또는 이진 탐색과 같은 말로 쓰지 않습니다.
- 힙을 중위순회해도 일반적으로 정렬되지 않습니다.

내용을 가리고 회상

1. 5의 왼쪽 자식 3 아래에 6이 있으면 BST입니까?

2. 두 자식 노드를 지울 때 후속자는 어디에서 찾습니까?

3. 오름차순 입력이 일반 BST 트리정렬에 불리한 이유는 무엇입니까?

적용 기준과 근거

2026-11-05 원출처 열람에 따른 강사 생성 설명·자체 예제입니다. 공식 기출 해설이 아닙니다. NIST treesort (1)의 BST 기반 의미를 사용하며 평균 복잡도의 입력 가정을 명시했습니다.

내용 검토 상태: 출처 확인. 출처 확인은 공식 정답이나 학습 성취를 뜻하지 않습니다.

출처별 상세 열람 범위는 위키의 출처 기록에서 확인합니다.

- Dictionary of Algorithms and Data Structures: binary search tree - NIST / Paul E. Black; 웹 항목; 수정일은 이번 열람에서 확인하지 않음; 확인 2026-11-05.
- Dictionary of Algorithms and Data Structures: treesort (1) - NIST / Paul E. Black; 2019-02-12 수정; 확인 2026-11-05.
- Algorithms, 4th edition, 3.2 Binary Search Trees - Robert Sedgewick / Kevin Wayne; Algorithms 4판 공개 교재; 페이지 날짜 미확인; 확인 2026-11-05.